Secure Document

Management System

Technical specification — ingestion, automated classification, and controlled access for PDF, Word, and Excel documents

---

Version 1.0 · August 2026

Scope: full-stack system design, intern project delivery

Contents

Page numbers omitted; use Word's navigation pane (View › Navigation Pane) to move between sections.

1 Technology stack

> **Deployment assumption.** This specification assumes a self-hosted deployment. A system whose purpose is classifying documents by sensitivity usually cannot send those documents to third-party APIs. This choice determines several stack selections: MinIO rather than S3, Keycloak rather than a hosted identity provider, and a large language model. The architecture is unchanged if that decision is reversed; only the endpoints move.

1.1 Backend

Python, because every capable document parser and every classification library lives there. A Node API layer is workable but ends up shelling out to Python regardless.

Concern	Choice	Rationale
API framework	FastAPI + Pydantic + Uvicorn	Async, typed, generates OpenAPI for the frontend client
Task queue	Celery + Redis	Parsing a 400-page PDF must never occupy a request handler
Object storage	MinIO (S3-compatible)	File bytes never belong in the database
Metadata database	PostgreSQL 16	Records, audit log, full-text via tsvector, vectors via pgvector
Search	Postgres FTS	Sufficient to six figures of documents; OpenSearch only past that
Identity	Keycloak (OIDC)	No per-user pricing cliff, self-hosted
Malware scanning	ClamAV	Runs on every upload before it leaves quarantine
Vector search	pgvector (HNSW index)	Same database as metadata; avoids a second datastore to secure
Reverse proxy	Nginx	TLS termination, body size limits, rate limiting
Observability	Structured JSON logs + Prometheus	Audit log is separate and non-negotiable; this is operational telemetry
Backups	pg_dump + object versioning	Database and blobs have different recovery paths and must both be tested

1.2 Document parsing

Format	Library	Notes
PDF (text layer)	PyMuPDF	Fast; returns layout coordinates needed for evidence highlighting
PDF (tables)	pdfplumber	Used selectively; slower than PyMuPDF
PDF (scanned)	OCRmyPDF / Tesseract	Fallback triggered only when the text layer is empty
DOCX	python-docx	—

Format	Library	Notes
XLSX	openpyxl	pandas for cell data analysis
Legacy .doc / .xls	LibreOffice headless	Convert first, then parse with the modern handler

An alternative is to wrap everything behind a single entry point — Apache Tika or unstructured. That is less code and less control, and it is a reasonable trade if parser tuning is not where the project's value lies.

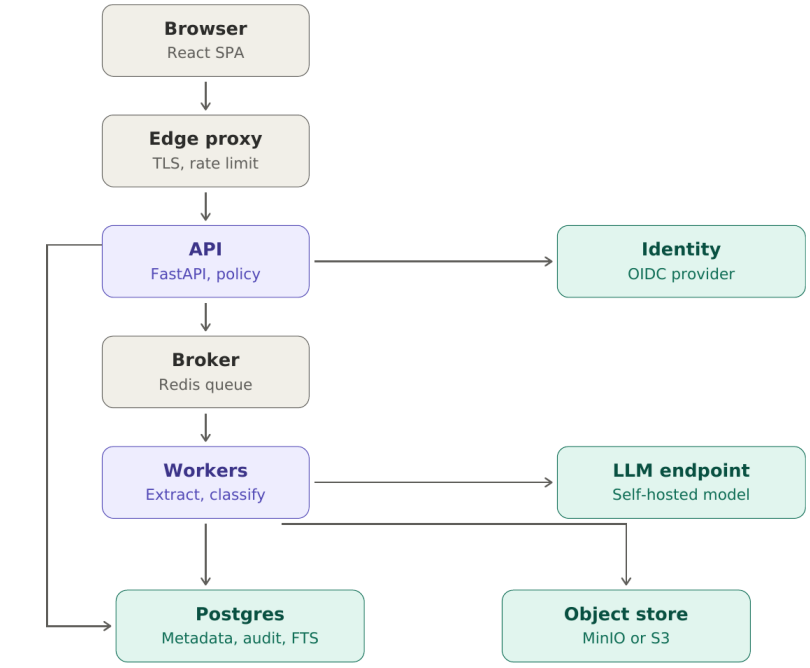
1.3 Classification

Layer	Technology	Role
Keyword extraction	spaCy NER + scikit-learn TF-IDF	Feeds rules, search facets, and the keyword index in one pass
Rules	Microsoft Presidio + custom recognisers	Security level; auditable, deterministic, no training data
Machine learning	sentence-transformers (bge-small) + scikit-learn	Document type; handles the distributional majority
Language model	Undecided	Zero-shot fallback for the ambiguous tail only
Layout model	LayoutLMv3 (optional)	Only if scanned forms are in scope

1.4 Frontend

Concern	Choice	Notes
Framework	React 18 + TypeScript + Vite	—
Server state	TanStack Query	Caching, background refetch, optimistic updates
Data grid	TanStack Table	Sorting, faceted filters, column sizing
Styling	Tailwind + shadcn/ui	—
Document preview	react-pdf (pdf.js)	Non-PDF formats rendered server-side to a single viewer path
Upload	Uppy / tus-js-client	Resumable; plain multipart fails on large files
Live status	Server-Sent Events	Processing progress without polling

2 System architecture



Purple denotes application code; teal denotes stateful or external services; grey denotes transport.

2.1 Notable properties

The API does not touch object storage on the write path. The browser uploads directly to storage using a presigned PUT; the API issues the signature and records intent. Large files never occupy a Python process. On the read path the API *does* stream bytes for restricted documents — the single asymmetry in the diagram, and a deliberate one (section 5.3).

Workers are the only automated writer of classifications. The API may record a human reclassification, but the machine path runs strictly broker → worker → database. Nothing classifies inside a request handler.

Identity is validated by the API against cached JWKS, not round-tripped to Keycloak per request.

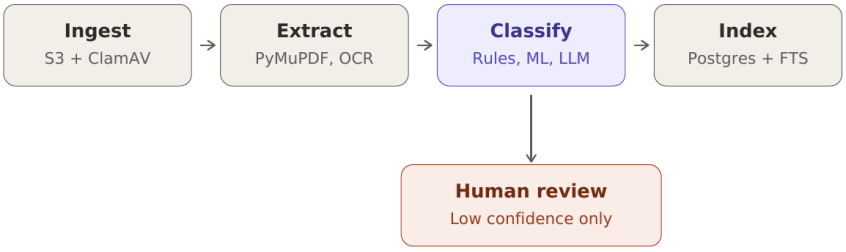
2.2 Trust boundaries

Boundary	Control
Edge → API	The only entry point for untrusted input. Body size limits, request timeouts, and rate limits belong here rather than in application code.
Workers → LLM	The only point at which document text leaves the process. If this endpoint were external, restricted content would cross the network perimeter — which is why it is self-hosted.
Any → Postgres	Row-level security applies regardless of caller. The worker connects under a different role than the API; neither holds DELETE on the audit log.

2.3 Deployment

Five containers for development — proxy, api, worker, postgres, minio — with Keycloak and the model server added as needed. The api and worker services scale horizontally and independently; the worker pool is the one that actually needs to grow, since parsing dominates wall-clock time. OCR runs on a separate Celery queue with its own pool, because a document taking minutes will otherwise starve everything else off a shared pool.

3 Classification system



Ingest pipeline. Low-confidence classifications branch to human review rather than being trusted.

3.1 Stage zero — keyword and entity extraction

Before any classifier runs, one pass over the extracted text produces named entities (spaCy NER) and weighted terms (TF-IDF against the corpus). This is not itself classification; it is shared input, and running it once rather than three times is what keeps the pipeline affordable.

Its output feeds three consumers:

- **The rules layer** uses recognised person and organisation entities as context signals — a salary figure near a detected person name scores far higher than one in a budget table.
- **The search index** gets a keyword row per document, which powers facets and gives the keyword half of hybrid retrieval something better than raw term frequency.
- **The reviewer** sees the top terms alongside the predicted label, which makes a wrong prediction diagnosable at a glance.

The embedding computed in section 3.3 is likewise generated once and stored, then reused for both classification and semantic search. Two passes over the same text with the same model is pure waste, and it is an easy mistake to make when search and classification are built by different people.

3.2 Layer one — rules, for security level

Regular expressions work here because sensitive identifiers are *structured*. A CNIC is thirteen digits in a known shape; an IBAN carries a checksum; a card number satisfies Luhn.

A bare pattern, however, is nearly useless on real documents. `\d{5}-\d{7}-\d` matches a CNIC and also matches an invoice line item that happens to be formatted the same way. A rule is therefore three things, not one:

```
class CNICRecognizer:
    pattern = r'\b\d{5}-\d{7}-\d\b'

    def validate(self, match):
        digits = match.replace('-', '')
        return digits[0] not in '069'      # invalid province code

    context_words = ['cnic', 'nic', 'identity', 'shanakhti', 'card no']
```

The pattern finds candidates, the validator eliminates structurally impossible ones, and context words within a ±50 character window raise the score. A match with no supporting context scores around 0.4; the same match beneath a heading reading “CNIC” scores 0.9.

Findings then aggregate into a label through a policy table — owned by whoever owns the security policy, not by engineering:

Finding	Level
Any card number, or three or more CNICs	Restricted
Salary figures alongside named individuals	Restricted
Single CNIC, or internal email domains	Confidential
Named employees, no identifiers	Internal
Nothing matched	Internal

The last row matters. Absence of evidence is not evidence of absence — an unclassifiable document defaults *upward*, never to Public.

Rules provide something neither other layer can: when an auditor asks why a document was marked Restricted, the answer is a named rule and a character offset.

3.3 Layer two — machine learning, for document type

Regular expressions fail at type because an invoice and a purchase order share nearly all their vocabulary. No string reliably separates them; the difference is distributional.

```
from sentence_transformers import SentenceTransformer
from sklearn.linear_model import LogisticRegression

model = SentenceTransformer('BAAI/bge-base-en-v1.5')

# the first two pages carry the signal: headers, parties, structure
X = model.encode([doc.text[:4000] for doc in train_docs])
y = [doc.doc_type for doc in train_docs]

clf = LogisticRegression(max_iter=1000, class_weight='balanced')
clf.fit(X, y)
```

Logistic regression rather than a fine-tuned transformer, for three reasons at this scale. It trains in seconds, permitting fast iteration. It works from a few hundred examples per class where fine-tuning wants thousands. And its outputs are approximately calibrated probabilities, whereas a fine-tuned classifier’s softmax is an overconfident score — which matters because routing decisions depend on that number.

Calibrate explicitly regardless:

```
from sklearn.calibration import CalibratedClassifierCV
clf = CalibratedClassifierCV(LogisticRegression(), method='sigmoid', cv=5)
```

A predict_proba of 0.7 then means the model is correct roughly 70% of the time at that confidence, which is what makes a threshold meaningful.

Layout models earn their cost only when field *position* carries meaning — scanned forms, invoices, applications. For born-digital contracts and reports, text embeddings perform as well at a fraction of the cost.

3.4 Layer three — language model, for the tail

Zero-shot, requiring no training data, and able to handle the document nobody anticipated. Output is constrained to a schema:

```
prompt = f"""Classify this document. Definitions:
{taxonomy_definitions}

Return only JSON: {{"doc_type": str, "confidence": float, "reasoning": str}}

Document excerpt:
{text[:6000]}"""
```

Its weaknesses are real: it is not reproducible run to run, it cannot be audited the way a rule can, it costs orders of magnitude more per document, and it will produce a confident label for a document it does not understand. Its self-reported confidence is ordinal at best.

It therefore never runs on everything, and it holds a constrained role: it may propose a document type, and it may *raise* a security level, but it can never lower one.

3.5 Combination

The two outputs merge by different rules.

Security level takes the maximum. Every layer contributes and the highest wins. This is monotonic by design; no later stage can talk the system down from Restricted. Only a human reviewer may lower a label, and that action is audited.

Document type cascades. The first confident answer wins:

```
def classify_type(doc):
    if r := rules_match(doc):          # unambiguous letterhead, form number
        return r, 1.0, 'rules'

    label, conf = ml_classify(doc)
    if conf >= 0.85:
        return label, conf, 'ml'

    label, conf = llm_classify(doc)
    if conf >= 0.75:
        return label, conf, 'llm'

    return None, conf, 'review'
```

Roughly 60–70% of documents settle at the machine learning layer, keeping language model cost proportional to the genuinely difficult tail rather than to total volume.

3.6 Evaluation

> **Errors are not symmetric.** A Restricted document misfiled as Internal is a data breach. An Internal document misfiled as Restricted is an inconvenience. A single accuracy figure averages these together and conceals the difference. Track per-class recall on the highest security label and hold it near 1.0, accepting whatever false-positive rate that costs. For document type, where errors are merely irritating, balanced accuracy is adequate.

Reserve 150–200 hand-labelled documents from the outset as an evaluation set and never train on them. Without a held-out set it is impossible to distinguish a model that improved from one that learned the labeller's habits.

3.7 Training data

Real personnel files cannot be used for development, so training data must come from elsewhere. The failure mode to avoid: generating documents by prompting a language model once per class produces a corpus with a signature — uniform length, uniform rhythm, recurring phrasings, clean structure. The classifier learns which prompt produced a document rather than what kind of document it is, scores 97% on its own test set, and collapses on real input.

For document type, use public corpora rather than generating. RVL-CDIP contains 400,000 scanned documents labelled across sixteen classes; Tobacco3482 is the 3,500-document subset that trains on a laptop. Also useful: SEC EDGAR (real filings by form type), CUAD (500+ annotated commercial contracts), FUNSD (scanned forms with field boxes).

For security level, generate — no public dataset labels documents by sensitivity, and real identifiers are unusable. The label is known before generation, so labelling is free:

```
from faker import Faker
fake = Faker('en_PK')

def make_cnic():
    province = fake.random_element(['1','2','3','4','5','7','8'])
    return f"{province}{fake.numerify('####')}-{fake.numerify('#######')}-{fake.random_digit()}"

SPECS = {
    'restricted':   {'cnic': (3, 8), 'salary': (2, 6), 'account': (1, 3)},
    'confidential': {'cnic': (1, 2), 'salary': (0, 1), 'account': (0, 0)},
    'internal':     {'cnic': (0, 0), 'salary': (0, 0), 'account': (0, 0)},
}
```

Vary four dimensions independently so the combinations multiply: **structure** (6–10 hand-written template skeletons per type), **surface form** (render across python-docx, openpyxl, and HTML-to-PDF, varying fonts and margins), **prose** (language model for body paragraphs only, with entities injected afterwards), and **degradation**.

Degradation matters most. Take 30% of generated PDFs and make them look scanned — slight rotation, Gaussian blur, sensor noise — then run them back through the OCR path so the training text contains genuine OCR errors:

```
def degrade(img):
    img = img.rotate(random.uniform(-1.5, 1.5), fillcolor='white')
    img = img.filter(ImageFilter.GaussianBlur(random.uniform(0.3, 0.9)))
    arr = np.array(img).astype(float)
```

```
    arr += np.random.normal(0, random.uniform(3, 9), arr.shape)
    return Image.fromarray(np.clip(arr, 0, 255).astype('uint8'))
```

Two further rules. **Strip label words** — a generated contract opens with “This Contract”, the classifier finds it, and learns nothing. **The evaluation set must contain real documents** — 50–100 genuine files, hand-labelled, never trained on. Report synthetic accuracy and real-document accuracy side by side; the gap between them is the meaningful result.

4 Search architecture

Search runs on two indexes over the same text, combined into one ranking. Keyword search finds documents containing the terms a user typed; semantic search finds documents about the thing they meant. Neither alone is adequate here — an employee searching “termination letter” should find a document titled “Notice of Separation”, and an auditor searching an exact policy number should not have it diluted by semantic neighbours.

4.1 The two indexes

```
document_text(
  version_id   uuid primary key references document_versions(id),
  tsv          tsvector,          -- GIN index
  embedding    vector(384),       -- HNSW index, cosine
  char_count   int,
  ocr_used     boolean
)
```

Both columns are populated by the same worker pass described in section 3.1, so neither index requires its own model invocation. The table is separate from documents because it is large, entirely regenerable, and will need reindexing when the embedding model changes — an operation that should never risk the metadata tables.

4.2 Fusion

The two result sets are merged with reciprocal rank fusion, which needs no score calibration between the two systems — a cosine distance and a ts_rank are not comparable numbers, and any attempt to weight them directly will need retuning every time either index changes.

```
WITH kw AS (
  SELECT version_id, row_number() OVER (ORDER BY ts_rank(tsv, q) DESC) AS r
  FROM document_text, plainto_tsquery($1) q
  WHERE tsv @@ q AND version_id = ANY($visible) LIMIT 100
),
vec AS (
  SELECT version_id, row_number() OVER (ORDER BY embedding <=> $2) AS r
  FROM document_text
  WHERE version_id = ANY($visible) LIMIT 100
)
SELECT version_id, sum(1.0 / (60 + r)) AS score
FROM (SELECT * FROM kw UNION ALL SELECT * FROM vec) t
GROUP BY version_id ORDER BY score DESC LIMIT 20;
```

> **The permission filter belongs inside both subqueries, not after the join.** Filtering a fused result set post-hoc is the single most common way a search feature leaks. It returns short pages that vary in length according to what the user cannot see, and total-hit counts become an oracle: repeated queries reveal how many Restricted documents match a given term, without a single one ever being displayed. Restrict the candidate set first, then rank what remains.

4.3 Snippets

Result snippets are generated from document_text, meaning a fragment of document content is rendered to anyone who can see the result. Since the candidate set is already permission-filtered this is safe by construction — but only because of that ordering. If the filter ever moves downstream, snippets become the leak rather than merely the counts.

4.4 Facets

Sidebar counts by security level, document type, and department come from the same filtered candidate set. A user must never see a facet count that includes documents they cannot open, for the same reason the total must not: a count is information.

5 Storage and access model

5.1 Bucket layout

Blobs live in object storage, everything else in Postgres. Three buckets by lifecycle stage:

```
docs-quarantine/{tenant}/{upload_id}          raw, unscanned, 24h TTL
docs-primary/{tenant}/{sha[0:2]}/{sha256}     immutable, content-addressed
docs-derived/{sha256}/text.json               extracted text + layout
docs-derived/{sha256}/page-0001.webp          preview renders
docs-derived/{sha256}/render.pdf              normalised docx/xlsx for the viewer
```

Content-addressing the primary bucket provides free deduplication, immutability, and single-operation integrity checking.

> **The object key is never an authorisation boundary.** Two tenants uploading the same NDA share one blob. Permission lives on the documents row, never on the object.

5.2 Versioning

Nothing in the primary bucket is ever overwritten. An edit produces a new blob and a new document_versions row pointing at it. Object Lock in governance mode, combined with soft deletes, provides legal hold without a separate archive system: a delete hides the row while the blob survives its retention window.

5.3 Read path

Presigned URLs are the conventional answer and they are wrong for the highest classification tier. A presigned URL is a bearer token in a query string: it can be pasted into a chat message, it survives session revocation, it appears in browser history and proxy logs, and the audit trail records *“URL issued”* rather than *“file was read.”* For a Restricted document that gap is the entire problem.

Access therefore splits by label:

Classification	Mechanism
Restricted, Confidential	GET /api/documents/{id}/content streams through the API. Range headers honoured so the viewer can seek. Every response is an audit row. Slower and it costs egress twice, but access is revocable mid-session and provably logged.
Internal, Public	Presigned URL with 60–120 second TTL, response-content-disposition pinned so the filename cannot be spoofed. Issuance is logged with document id and actor.

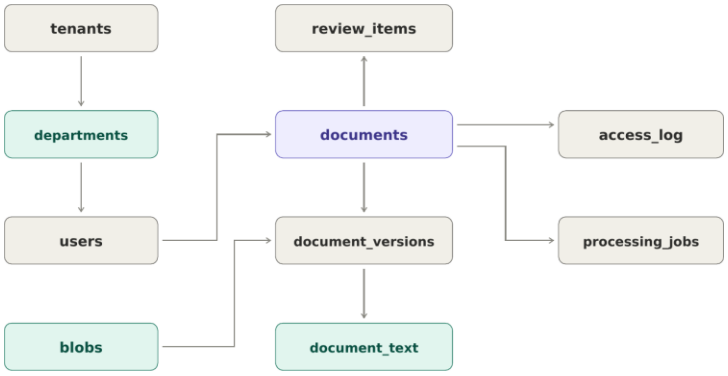
5.4 Preview versus download

These are separate privileges. The viewer renders page-N.webp from the derived bucket; original bytes never move. Retrieving the actual file is a distinct endpoint, a distinct permission check, and the audit event that matters at review time. A user who can read a contract on screen does not automatically obtain a copyable original.

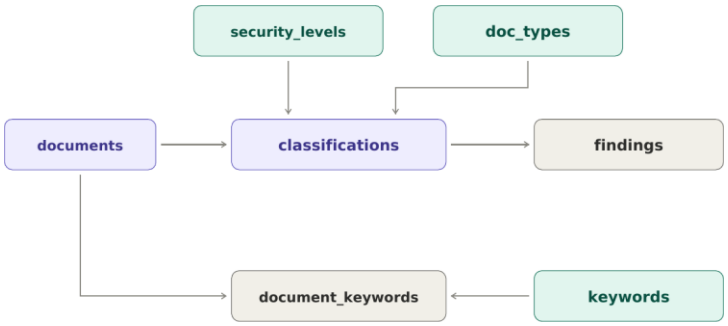
5.5 Operational details

- **Sniff the MIME type; never trust the extension.** An .xlsx that is actually an HTML file with a formula payload is a real attack.
- **Encrypt per tier.** A distinct KMS key per classification level means revoking an entire tier is a policy change rather than a bucket crawl.
- **Lifecycle rules keyed on classification.** Tag objects with their label at write time; Public documents may drop to infrequent access after 90 days, Restricted documents generally cannot leave the hot tier.

6 Database schema



Document, storage, and processing tables. Arrows point from parent to child.



Classification, taxonomy, and keyword tables.

```
tenants(id uuid pk, name text, created_at timestamptz)

departments(id uuid pk, tenant_id uuid fk, parent_id uuid fk null,
            name text, created_at timestamptz)

users(id uuid pk, tenant_id uuid fk, department_id uuid fk, oidc_sub text unique,
      email text, role text, clearance_rank smallint, created_at timestamptz)

blobs(sha256 text pk, size_bytes bigint, mime_sniffed text,
      bucket_key text, created_at timestamptz)

documents(id uuid pk, tenant_id uuid fk, department_id uuid fk, original_filename text,
          status text,              -- quarantined | processing | ready | failed | held
          current_classification_id uuid fk deferrable,
          uploaded_by uuid fk, created_at timestamptz, deleted_at timestamptz)

document_versions(id uuid pk, document_id uuid fk, blob_sha256 text fk,
                  version_no int, created_by uuid fk, created_at timestamptz,
                  unique (document_id, version_no))

security_levels(id uuid pk, rank smallint unique, name text, description text)

doc_types(id uuid pk, parent_id uuid fk null, name text, description text,
          unique (parent_id, name))           -- category / subcategory

processing_jobs(id uuid pk, document_id uuid fk, version_id uuid fk,
                stage text,   -- scan | extract | keywords | embed | classify | index
                state text,   -- queued | running | succeeded | failed
```

```
            attempts int, error text, started_at timestamptz, finished_at timestamptz)

keywords(id uuid pk, term text unique, idf real)

document_keywords(document_id uuid fk, keyword_id uuid fk, score real,
              primary key (document_id, keyword_id))

document_text(version_id uuid pk fk, tsv tsvector, embedding vector(384),
           char_count int, ocr_used boolean)

classifications(id uuid pk, document_id uuid fk, version_id uuid fk,
             level_id uuid fk, doc_type_id uuid fk, confidence real,
             decided_by text,          -- rules | ml | llm | human
             created_at timestamptz)

findings(id uuid pk, classification_id uuid fk, entity_type text, rule_id text,
       page_no int, char_start int, char_end int, score real)

review_items(id uuid pk, document_id uuid fk, assigned_to uuid fk,
          state text, created_at timestamptz, resolved_at timestamptz)

access_log(id bigserial pk, document_id uuid, actor_id uuid, action text,
         ip inet, user_agent text, ts timestamptz)
```

6.1 Design decisions

No classification columns on documents. The row holds current_classification_id and nothing else about the label. Carrying a denormalised classification or confidence alongside an append-only classifications table creates two sources of truth that drift the first time a write partially fails — and the one that drifts is the one governing access.

Blob attributes live on blobs, not on documents. Size, sniffed MIME type, and hash describe stored bytes, not a logical document. Placing them on documents makes them silently wrong the moment a second version exists. A document reaches its bytes through document_versions.

A classification references a version, not just a document. Without version_id, uploading new content leaves the previous verdict attached and looking current — a document could be re-uploaded with sensitive material and retain its Internal label until something reclassified it.

The documents and classifications foreign keys are mutually circular — the document points at its current classification and the classification points back at its document. Declare current_classification_id as DEFERRABLE INITIALLY DEFERRED so both rows can be written in one transaction; otherwise no ordering satisfies both constraints.

blobs is separate from documents. Two people uploading the same policy PDF produce one blob and two document rows. The sha256 is the natural primary key, so deduplication falls out and integrity checking is a rehash. Critically, blobs carries no tenant and no permission.

classifications is append-only. A document does not *have* a classification; it has a history of them. documents.current_classification_id points at the live one and every prior decision persists. This yields “who reclassified this and when” without a second audit table, and it makes shadow-mode evaluation trivial — write the model’s prediction as a row nobody points at, then compare.

security_levels carries a surrogate id plus a separate rank. Making rank the primary key means that inserting a level between two existing ones requires renumbering every foreign key. Keeping them separate lets the ordinal drive the monotonicity rule:

```
CREATE FUNCTION check_monotonic() RETURNS trigger AS $$
BEGIN
  IF NEW.decided_by <> 'human' AND EXISTS (
    SELECT 1 FROM classifications c
    JOIN security_levels old ON old.id = c.level_id
    JOIN security_levels new ON new.id = NEW.level_id
    WHERE c.document_id = NEW.document_id AND new.rank < old.rank
  ) THEN
    RAISE EXCEPTION 'automated reclassification cannot lower security level';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Enforcing this in the database rather than the service layer means a defective worker or a stray script cannot quietly downgrade a document.

findings answers the question “why”. Each row records one rule firing at one character offset with one score. Store offsets rather than matched text, so identifiers are not duplicated into a second table.

Access is gated on two independent axes. Clearance rank answers *how sensitive* a document is; department answers *whose business* it is. Neither subsumes the other — a senior manager with Restricted clearance has no business reading another division’s disciplinary files, and a junior HR clerk needs department access to documents well above the clearance of most staff. Collapsing the two into a single rank is the mistake that forces a rewrite later.

Departments are hierarchical via parent_id, so visibility is a subtree rather than a single node:

```
WITH visible_depts AS (
  SELECT id FROM departments WHERE id = $3
  UNION ALL
  SELECT d.id FROM departments d JOIN visible_depts v ON d.parent_id = v.id
)
SELECT d.* FROM documents d
JOIN classifications c ON c.id = d.current_classification_id
JOIN security_levels sl ON sl.id = c.level_id
WHERE d.tenant_id = $1
  AND sl.rank <= $2                          -- caller's clearance
  AND (d.department_id IS NULL
       OR d.department_id IN (SELECT id FROM visible_depts))
  AND d.deleted_at IS NULL;
```

Wrap this in row-level security rather than trusting every query to remember the tenant filter; one forgotten WHERE clause is otherwise a cross-tenant leak.

access_log must not cascade. Declare the foreign key ON DELETE NO ACTION, or drop the constraint and keep document_id as a bare uuid. An audit trail that disappears alongside the thing it audits is not an audit trail. For the same reason the application role holds no UPDATE or DELETE grant on it.

processing_jobs makes pipeline state a database fact rather than a Celery implementation detail. Without it, “why has this document been processing for an hour?” requires reading broker internals, the Uploads screen has nothing to render, and a stage that failed three times looks identical to one that never started. One row per stage per version, with attempt count and error text.

doc_types is hierarchical rather than flat. A parent_id gives category and subcategory in one table — Contract › Vendor MSA, HR › Disciplinary. The classifier predicts leaves; filters and facets can roll up to parents. Two separate columns for category and subcategory would make adding a third level a migration rather than an insert.

keywords is global, document_keywords is the join. Storing the term once with its corpus IDF, rather than repeating strings per document, keeps the table small enough to drive facet queries directly and means recomputing IDF touches one table.

6.2 Indexes

```
CREATE INDEX ON documents (tenant_id, status) WHERE deleted_at IS NULL;
CREATE INDEX ON document_versions (document_id, version_no DESC);
CREATE INDEX ON classifications (document_id, created_at DESC);
CREATE INDEX ON access_log (document_id, ts DESC);
CREATE INDEX ON access_log (actor_id, ts DESC);
CREATE UNIQUE INDEX ON document_versions (document_id, version_no);
CREATE INDEX ON documents (tenant_id, department_id) WHERE deleted_at IS NULL;
CREATE INDEX ON processing_jobs (document_id, stage);
CREATE INDEX ON processing_jobs (state) WHERE state IN ('queued', 'running');
CREATE INDEX ON document_text USING GIN (tsv);
CREATE INDEX ON document_text USING hnsw (embedding vector_cosine_ops);
CREATE INDEX ON document_keywords (keyword_id, score DESC);
```

The partial index on documents matters more than it appears: nearly every list query excludes soft-deleted rows, and keeping them out of the index keeps it small.

The partial index on processing_jobs matters for the worker: the queue-depth query runs constantly and should never scan completed jobs, which will outnumber live ones by orders of magnitude within weeks.

7 Backend structure

Two processes, one codebase, one image. The API and the Celery worker differ only in entrypoint command; they import the same domain code, which prevents the classification rules from silently forking into two versions.

```
backend/
  app/
    main.py              FastAPI app, middleware, router mounting
    config.py            pydantic-settings, env-driven
    deps.py              DI: db session, current_user, storage client

    api/v1/
      uploads.py          POST /uploads, presigned init, completion
      documents.py          list, detail, content, reclassify
      review.py           queue, claim, resolve
      search.py           hybrid query, facets
      audit.py           read-only log endpoints
      admin.py            taxonomy CRUD, metrics
      errors.py          exception to RFC 7807 handlers

    domain/
      models.py           pydantic domain objects (not ORM rows)
      policy.py          authorization + level aggregation
      taxonomy.py           level/type definitions, loaded from DB

    db/
      models.py           SQLAlchemy tables
      repositories/       documents, classifications, audit
      migrations/         alembic

    storage/
      base.py            Storage protocol
      s3.py             boto3 / MinIO
      local.py           filesystem, for tests and dev

    extraction/
      registry.py          mime to handler dispatch
      pdf.py  docx.py  xlsx.py  ocr.py
      keywords.py           spaCy NER + TF-IDF, one pass

    search/
      hybrid.py           reciprocal rank fusion
      filters.py          permission predicate, applied pre-ranking

    classification/
      pipeline.py          orchestrates the three layers
      rules/             recognizers.py, aggregate.py
      ml/                embed.py, classifier.py, train.py
      llm/               client.py, prompts.py

    security/
      auth.py            OIDC token validation, JWKS cache
      permissions.py        FastAPI dependency wrappers
      audit.py           audit-write helper

    workers/
      celery_app.py
      tasks.py            scan, extract, keywords, embed, classify, index
```

```
    jobs.py              processing_jobs state transitions

tests/
pyproject.toml
```

7.1 Key seams

domain/policy.py is the single source of truth. Both the API and the worker import it; nothing else decides a security level or an access grant.

```
def can_access(user: User, doc: Document, action: Action) -> bool:
    if user.tenant_id != doc.tenant_id:
        return False
    if doc.deleted_at and action != Action.RESTORE:
        return False
    if user.clearance_rank < doc.level_rank:          # how sensitive
        return False
    if doc.department_id is None:
        return True
    return doc.department_id in user.visible_departments   # whose business

def aggregate_level(findings: list[Finding], taxonomy: Taxonomy) -> Level:
    """Highest level implied by any finding. Never returns below the floor."""
    ranks = [taxonomy.rank_for(f) for f in findings]
    return taxonomy.by_rank(max(ranks, default=taxonomy.default_floor))
```

Both functions are pure — no database, no request object, no session. The authorisation test suite is therefore a parametrised table over roles and levels, running in milliseconds with no fixtures.

Storage is a Protocol, not a boto3 import. Local development and CI run against the filesystem implementation, so nobody needs MinIO installed to run tests.

```
class Storage(Protocol):
    def put(self, key: str, data: BinaryIO, *, content_type: str) -> str: ...
    def open(self, key: str, *, byte_range: tuple[int, int] | None = None) -> BinaryIO: ...
    def presign(self, key: str, ttl: int, *, filename: str) -> str: ...
```

Extraction dispatches on sniffed MIME type. Each handler returns the same ExtractedDocument shape regardless of source format, so the classification package never learns what a docx is.

7.2 API surface

Method	Path	Notes
POST	/v1/uploads	Init; returns upload id and presigned PUT
POST	/v1/uploads/{id}/complete	Enqueues the processing pipeline
GET	/v1/documents	Facets, cursor pagination
GET	/v1/documents/{id}	Metadata and current classification
GET	/v1/documents/{id}/content	Proxy stream or 302, depending on level

Method	Path	Notes
GET	/v1/documents/{id}/pages/{n}	Preview render
GET	/v1/documents/{id}/findings	Evidence panel data
POST	/v1/documents/{id}/classification	Human reclassification
GET	/v1/search	Hybrid query; facets returned alongside results
GET	/v1/documents/{id}/jobs	Per-stage processing state
GET	/v1/review	Review queue
POST	/v1/review/{id}/resolve	Accept or correct
GET	/v1/audit	Filter by actor, document, action
GET	/v1/events	Server-sent events, processing status

Cursor pagination rather than offset: LIMIT 50 OFFSET 10000 on a filtered document table becomes noticeably slow and double-shows rows when a document is reclassified mid-scroll.

7.3 Worker chain

```
@shared_task(bind=True, max_retries=3, autoretry_for=(TransientError,))
def process(self, upload_id: str):
    chain(
        scan_for_malware.s(upload_id),
        extract_text.s(),          # OCR fallback when text layer is empty
        extract_keywords.s(),      # spaCy NER + TF-IDF, feeds rules and search
        embed.s(),                 # one pass, reused by classify and search
        classify.s(),
        build_index.s(),           # tsvector + vector, both from stored text
    ).apply_async()
```

Each task is idempotent and keyed on the blob hash, so a retry after a worker dies mid-chain does not produce a duplicate classification row. Every stage writes its transition to processing_jobs before and after running, which is what makes the pipeline observable from SQL rather than only from the broker.

> **Two decisions to make before writing code.** First, audit writes belong in the same transaction as the action they record — not a middleware, not a background task. If a download commits and its audit row does not, there is a silent hole exactly where one is least wanted. Second, an error that leaks filenames is a real leak: a 404 for a document in another tenant must be indistinguishable from a 404 for a document that never existed, in body and in timing. Centralise this in errors.py so no endpoint gets to be creative about it.

8 Frontend structure

```
src/
  api/          generated client, TanStack Query hooks
  features/
    documents/  grid, filters, detail, viewer
    review/     queue, keyboard bindings
    upload/     dropzone, progress, retry
    audit/
    admin/
  components/   shadcn primitives + shared
  lib/          auth context, permissions, formatters
```

8.1 Routes

```
/documents                  library, facets in URL params
/documents/:id              detail - viewer + metadata + evidence
/documents/:id/audit          who touched this file
/review                   queue, keyboard-driven
/search                   hybrid results, facets in URL params
/uploads                   in-flight processing status, per-stage
/audit                    global log, filter by actor or action
/admin/taxonomy             levels, types, rule definitions
/admin/metrics              classifier performance
```

Filter state belongs in the URL. Someone will want to send a colleague a link to “all Restricted HR records.”

8.2 Main screen layout

A left sidebar carries navigation and classification facets with live counts; the main panel holds search and a document table. The counts are the point — a user glancing at “Restricted: 21” learns something about the corpus before clicking anything, and an unexpected spike becomes visible.

Needs review appears in the level column rather than as a separate status field. It is not a fourth security level; it is the absence of a confident one, and placing it where the label belongs prevents anyone mistaking an unreviewed file for a cleared one.

8.3 The two screens requiring real thought

Document detail uses three columns: page thumbnails, the rendered page, and a metadata plus evidence panel. Each finding renders as “CNIC detected · page 3 · rule cnic_v2 · score 0.91”; clicking one scrolls the viewer to that page and highlights the region. This is the feature that converts “the computer said so” into something a compliance officer will accept.

The review queue should be built for speed rather than appearance. One document, its predicted labels, the evidence, and keyboard shortcuts: A to accept, number keys to select a type, arrow keys to adjust the level, Enter to advance. Whoever reviews 300 documents to build the training set will use this more than anyone uses the library, and a queue costing four clicks per item will quietly destroy labelling throughput.

8.4 Permissions in the client

Put the check in a <Can level={doc.level}> wrapper and a usePermissions() hook, used everywhere. Scattering user.role === ‘admin’ through components is how the UI and the API drift apart. The client-side check is cosmetic regardless — the server decides, and the client only hides buttons it knows would fail.